毫不留情地说：**这个解法目前不能作为数学建模竞赛论文的正式求解依据**。它看起来像“很努力地写了一个 CP-SAT 调度器”，但核心建模假设、数据读取、问题4、运输时间、输出表结构都有严重硬伤。最危险的是：它可能给出一个看似漂亮的最短时长，但这个时长**很可能不是题目真实约束下的可行解**。

我按严重程度说。

---

1. 最大硬伤：没有读取附件，直接硬编码数据

代码开头直接写死了：

```
operations = {...}
team1_machines = [...]
team2_machines = [...]
distances_raw = {...}
machine_prices = {...}
```

这在数学建模比赛里是非常危险的。题目明确要求“结合附件数据”，而这个程序没有真正读取 `B-附件.xlsx`。如果附件里任意一个数、单位、工序结构、设备速度、表头解释和硬编码不一致，后面所有结果都作废。

更严重的是：你论文里如果写“读取附件数据”，但代码实际上硬编码，属于**可复现性失败**。评委或老师一看代码就会质疑：这些数怎么来的？有没有抄错？有没有漏掉？有没有人为改造？

必须改： 用 `pandas.read_excel()` 读取三个 sheet，并自动构建 `operations` / `machines` / `distance`。硬编码最多只能作为临时调试，不能作为正式解法。

---

2. 运输时间建模是残缺的，甚至可能让结果严重偏小

题目里跨车间运输时间是核心约束。但这个程序有三个致命问题。

2.1 没有计入设备从班组初始位置到第一个车间的运输时间

CP-SAT 模型只处理了设备在两个工序之间跨车间切换时的运输时间：

```
if ws_i == ws_j:
    trans_time = 0
else:
    trans_time = get_transport_time(ws_i, ws_j)
```

但是设备第一次出发时，应该从“班组1”或“班组2”移动到对应车间。题目给了班组到车间距离，显然就是要用的。代码在 CP-SAT 里完全没有加：

```
start_op >= transport_time(team, workshop)
```

这意味着设备可以在 `t=0` 瞬间出现在 A/B/C/D/E 任意车间开工。这个错误会让所有问题的 makespan 偏小，尤其是问题2、3、4。

这是**一级致命错误**。

2.2 设备移动速度被硬编码成 2.0，完全忽略附件中的设备移动速度

代码写的是：

```
def calc_transport_time(distance, speed=2.0):
    return math.ceil(distance / speed)
```

但题目说“班组配置表记录设备移动速度”。不同设备移动速度可能不同，必须按设备编号读取。现在所有设备都默认 2.0，这相当于把题目给的移动速度列废掉了。

这会直接影响跨车间转运时间，进而影响最优调度结果。

正确形式应该是：

```
tau = ceil(distance[from_loc, to_loc] / machine_speed[machine_id])
```

而不是统一 `distance / 2.0`。

2.3 可行性检查根本没有检查运输时间

`check_feasibility()` 只检查了：

- 是否所有工序调度；
- 工序顺序；
- 设备不重叠；
- 双设备工序是否同一开始。

但它**没有检查同一设备前后两道跨车间工序之间是否留足运输时间**。也没有检查首次从班组到车间的运输时间。

所以即使 CP-SAT 模型或启发式结果违反运输约束，检查也可能显示“通过”。

这会导致论文中“可行性验证”部分失真。

---

3. 双设备协同工序的处理过于武断，可能错解题意

代码把双设备工序强制为两类设备**同一开始时间**：

```
op_start[op_id]
```

所有设备类型共用一个 `op_start`。

但题目说：

> 不考虑两类设备的先后顺序，不考虑两类设备之间的等待时间，两类设备都需要完成该工序对应的工程量，两类设备都完成后，该工序才可判定为完成。

这更合理的解释是：两类设备可以分别在满足前序工序完成和设备可用后开工；工序完成时间取两者完成时间最大值。并不必然要求两台设备同一时刻开始。

如果强制同步开始，会造成两种问题：

一方面，模型可能过于保守，错过更优排程；另一方面，它把“等待时间不计”误解成“必须同时开始”。这不是自然的调度建模方式。

更合理的变量应该是：

```
S_{j,k1}, C_{j,k1}
S_{j,k2}, C_{j,k2}
C_j = max(C_{j,k1}, C_{j,k2})
后续工序 start >= C_j
```

而不是所有设备都用同一个 `op_start[j]`。

---

4. C 车间工序顺序处理高度可疑，而且 CP-SAT 与贪心算法前后不一致

代码中 C 车间写成：

```
"C": ["C1", "C2", "C3_1", "C4_1", "C5_1", "C3_2", "C4_2", "C5_2", "C3_3",
"C4_3", "C5_3"]
```

但在 CP-SAT 里，工序顺序不是按这个列表，而是按 `order` 排序：

```
workshop_ops.sort(key=lambda x: operations[x]["order"])
```

由于 `C3_1`、`C3_2`、`C3_3` 的 `order` 都是 3，排序后很可能变成：

```
C1 → C2 → C3_1 → C3_2 → C3_3 → C4_1 → C4_2 → C4_3 → C5_1 → C5_2 → C5_3
```

但贪心启发式用的是你手写的：

```
C1 → C2 → C3_1 → C4_1 → C5_1 → C3_2 → C4_2 → C5_2 → C3_3 → C4_3 → C5_3
```

也就是说，**CP-SAT 和 greedy 对 C 车间的工序先后关系理解不一致。**

这是非常严重的问题，因为如果 C3、C4、C5 是多组重复链，正确关系可能是：

```
C1 → C2
然后：
C3_1 → C4_1 → C5_1
C3_2 → C4_2 → C5_2
C3_3 → C4_3 → C5_3
```

也可能是所有 C3 完成后再 C4，再 C5。必须从附件表结构确认，不能靠硬编码猜。

现在这段代码对 C 车间的处理很可能既不符合题意，也不自洽。

---

5. 问题4基本不合格：购置方案没有分配到班组1/班组2

题目表5要求：

设备名称	班组1购买台数	班组2购买台数

但代码的购买方案只有总数：

```
plan = {
    "自动化输送臂": auto_count,
    "工业清洗机": wash_count,
    ...
}
```

没有区分买给班组1还是班组2。

然后新购设备命名成：

```
f"{mtype}_新_{count}"
```

并被塞进：

```
type_to_machines[mtype]["新购"]
```

这相当于创造了一个不存在的“新购班组”。但题目明确说“用于各班组补增各类设备资源”，不是创建第三个资源池。

更糟的是：

```
def get_machine_team(machine_name):
    ...
    return "班组2"
```

所有不匹配班组1命名规则的设备，默认都被当成班组2。新购设备在某些地方又显示为“新购”。这会导致：

- 新设备初始位置不明确；
- 新设备所属班组不明确；
- 表5无法回填；
- 运输时间无法正确计算；
- 购买方案不符合题目格式。

问题4目前不能用。

---

6. 问题4外层搜索有任意截断，不能保证最优

代码限制每类设备最多买 5 台：

```
max_counts[mtype] = min(max_counts[mtype], 5)
```

自动传感多功能机和高速抛光机又被限制最多 3 台：

```
range(0, min(max_counts["自动传感多功能机"], 3) + 1)
```

这是人为拍脑袋截断。题目预算 500000 元，某些便宜设备理论上可以买超过 5 台。你可以基于“已有设备数量足够、瓶颈不在该类设备”去剪枝，但不能无证明地硬截断。

更严重的是，代码先用贪心启发式筛选前 10 个方案，再对前 10 个用 CP-SAT：

```
heuristic_results[:top_n]
```

这并不能保证 CP-SAT 最优方案一定在贪心排名前 10。贪心评价很可能错杀真正最优购置方案。

因此问题4最多是“启发式尝试”，不能写成“最优购置方案”。

---

7. 表4、表5输出反了

题目要求：

- 表4：问题4调度结果；
- 表5：设备购买情况。

代码中却是：

```
print("表4: 最优购置方案")
print("表5: 最优方案调度结果")
```

这直接和题目要求冲突。正式提交时这是非常低级但致命的格式错误。

---

8. 输出表缺少题目要求的 HH:MM:SS 时间格式

题目表格要求起始时间、结束时间类似 `00:00:00`。代码表格里只有：

```
"起始时间(s)"
"结束时间(s)"
```

虽然有 `format_time()` 函数，但没有把它用于表1—表5的正式输出。

正式结果表应该至少包括：

```
起始时间 = HH:MM:SS
结束时间 = HH:MM:SS
持续工作时间(s) = 秒数
```

现在的表无法直接回填题目模板。

---

9. 下界计算是错的，验证意义很弱

关键路径下界这里：

```
min_pt = min(calc_processing_time(op["workload"], op["efficiency"][t])
             for t in op["required_types"])
```

对于双设备工序，工序完成至少要两类设备都完成。若两类设备并行，工序最短持续时间应是：

```
max(p_type1, p_type2)
```

而不是：

```
min(p_type1, p_type2)
```

所以关键路径下界被低估。

设备负载下界也只是简单按类型平均，没有考虑：

- 双设备工序对两类设备同时占用；
- 班组限制；
- 初始运输；
- 跨车间运输；
- 车间内工序先后关系。

作为粗下界可以，但不能用来强力证明结果优良。

---

10. 可行性检查非常不完整

目前可行性检查没有检查：

- 设备类型是否匹配；
- 跨车间运输时间是否满足；
- 首次从班组到车间运输时间是否满足；
- 问题4预算是否满足；
- 新购设备是否属于班组1或班组2；
- 双设备工序是否两台设备都完成完整工程量；
- 表格中持续工作时间是否正确；
- 同 order 多工序的真实前后关系是否正确。

这意味着“可行性检查通过”没有太大可信度。

---

11. 代码工程层面也不干净

几个典型问题：

11.1 `pandas.read_excel` 没用上

导入了 pandas，但没有读取附件。形式上像完整程序，实质上还是硬编码脚本。

11.2 `hint_solution` 参数完全没用

```
hint_solution=None
```

但后面没有任何 `AddHint()`。这是遗留接口。

11.3 `team_assignments` 几乎没实际用

有参数，但四个问题基本没传。问题4也没有真正按班组购买设备。

11.4 输出路径写死

```
output_dir = "/mnt/agents/output"
```

如果不是在特定运行环境下，直接报错或输出到错误位置。

11.5 运行时间可能非常长

我尝试运行脚本，30 秒内没有完成。对于比赛现场，代码必须能稳定快速跑出结果，不能靠不确定的长时间 CP-SAT 搜索。

总体评价

这份代码的优点是：

1. 试图用 CP-SAT 表达柔性调度问题；
2. 考虑了设备不可重叠；
3. 尝试处理双设备工序；
4. 尝试加入运输时间；
5. 有甘特图和 Excel 导出意识。

但这些优点不足以抵消核心问题。

当前版本的真实水平：

维度	评价
数据可信度	不合格，硬编码，未读取附件
运输时间建模	不合格，缺首次运输，速度硬编码
双设备工序建模	有争议，过度强制同步
问题1	可能可用，但仍缺初始运输与附件校验
问题2	不可靠
问题3	不可靠
问题4	基本不可用
可行性检查	严重不足
结果表格式	不符合题目
论文可提交性	不能直接提交

一句话总结：

这是一个“看起来像调度优化器”的半成品，不是一个可以支撑特奖级论文的可靠求解器。

最优先修改清单

按优先级改，不要一上来美化图表。

第一优先级：重做数据层

必须实现：

```
read_excel(file_path, sheet_name="工序流程表")
read_excel(file_path, sheet_name="班组配置表")
read_excel(file_path, sheet_name="车间距离表")
```

并自动生成：

```
operations
machines
distances
machine_speed
machine_price
```

不要硬编码。

第二优先级：修正运输时间

每台设备必须有：

```
current_location
machine_speed
```

首次任务：

```
start >= ceil(distance[team_location, workshop] / speed[machine])
```

后续任务：

```
next_start >= previous_end + ceil(distance[prev_workshop, next_workshop] /
speed[machine])
```

并且可行性检查必须重新验证这一点。

第三优先级：重写双设备工序模型

不要强制同一开始时间。应该允许每类设备有自己的开始和结束时间：

```
S[j, type1], C[j, type1]
S[j, type2], C[j, type2]
C[j] = max(C[j, type1], C[j, type2])
```

后续工序：

```
S[next] >= C[j]
```

这样更符合调度逻辑，也可能得到更优结果。

第四优先级：确认 C 车间工序结构

必须从附件中解析 C 车间到底是：

1. 严格线性序列；
2. 三组并行子链；
3. 同 order 工序可并行；
4. 还是表格行本身就是严格顺序。

这决定整个调度模型。如果这里错了，后面全错。

第五优先级：重做问题4

问题4购买变量必须是：

```
z[mtype, team] >= 0, integer
```

而不是只买总数。

输出必须是：

```
设备名称 | 班组1购买台数 | 班组2购买台数
```

新购设备必须命名为：

```
自动化输送臂1-new-1
自动化输送臂2-new-1
```

并继承对应班组初始位置。

第六优先级：修正输出表

所有表必须符合题目格式：

```
序号
设备编号
起始时间 HH:MM:SS
结束时间 HH:MM:SS
持续工作时间(s)
工序编号
班组
```

问题4注意：**表4是调度表，表5是购买表**，现在反了。

如果你想继续推进，我建议下一步不要让 AI “润色这个代码”，而是直接让它按上述六项**重构求解器**。这份代码局部修补会越补越乱，尤其是问题4和运输时间，应该重写核心调度数据结构。